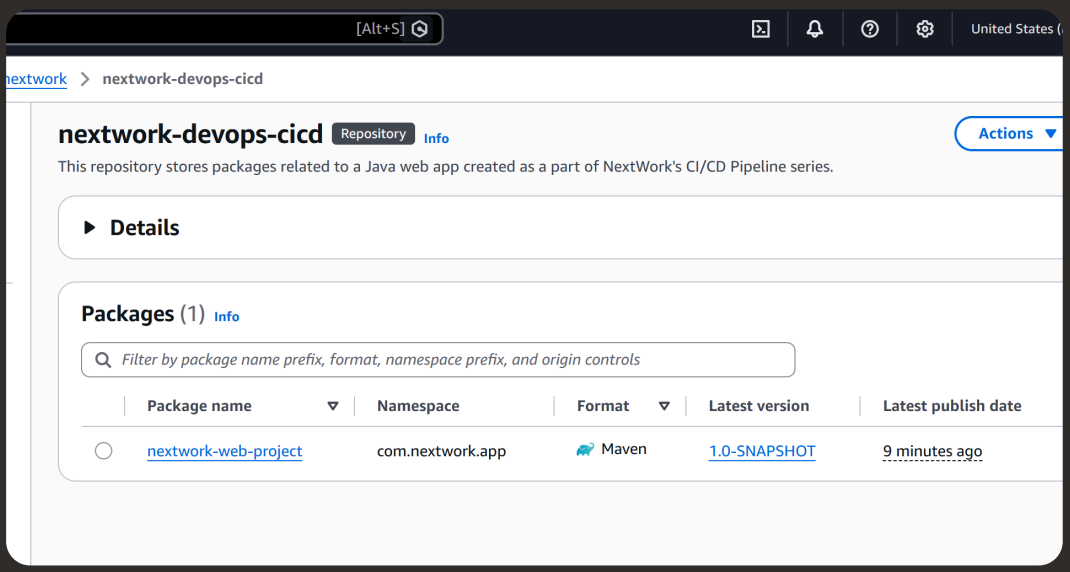
SECURE PACKAGES WITH CODEARTIFACT

By Maryna Maksimuk | October 2026



Introducing Today's Project!

Today, I will create an AWS CodeArtifact repository, configure my EC2 instance with the proper permissions to access it, and then use CodeArtifact to store and manage my web application's packages and dependencies.

Key tools and concepts

The key services and concepts I learned in this project were AWS CodeArtifact, Amazon EC2, IAM roles and permissions, Maven, and artifact management. I learned how to compile a Java web application on an EC2 instance using Maven and publish the application artifact to an AWS CodeArtifact repository. I also learned how EC2 uses IAM roles to authenticate with AWS services and how permissions control actions such as publishing and accessing packages.

Another important concept I learned was how Maven uses pom.xml and settings.xml to configure repositories and authentication. I also gained hands-on experience troubleshooting AWS permissions and Maven deployment errors, including a 403 Forbidden error, and learned how to verify that an application package was successfully published in CodeArtifact.

Project reflection

It took me approximately 2 hours to complete this project. During that time, I set up the Java web application, configured Maven and AWS CodeArtifact, compiled the application, and published the artifact to the repository.

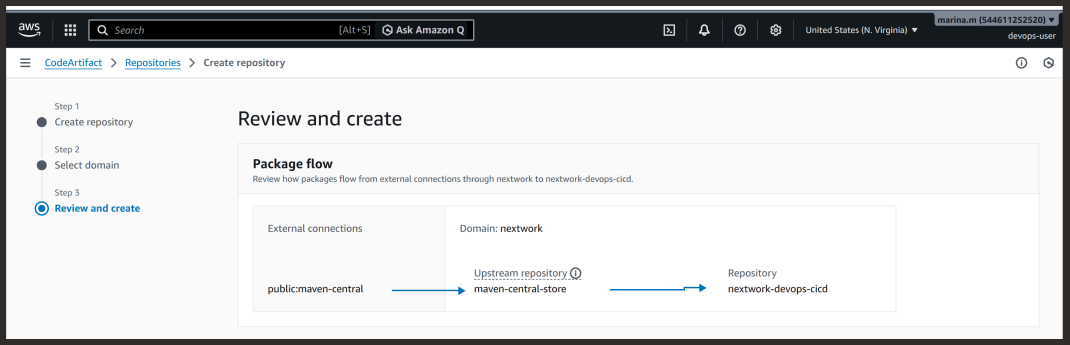
I plan to complete my next project tomorrow as part of my continued DevOps and AWS learning.

CodeArtifact Repository

AWS CodeArtifact is a managed service for securely storing, managing, and retrieving software packages and dependencies used by applications.

A CodeArtifact domain acts as a central container that lets me organize and manage multiple repositories in one place, while allowing them to share packages and configuration.

My upstream repository is Maven Central, which CodeArtifact uses as an external source to retrieve Java/Maven packages that are not already stored in my CodeArtifact repository.



CodeArtifact Security

Issue

We need an authorization token to securely authenticate Maven with CodeArtifact and verify that the EC2 instance has permission to access and download packages from the repository.

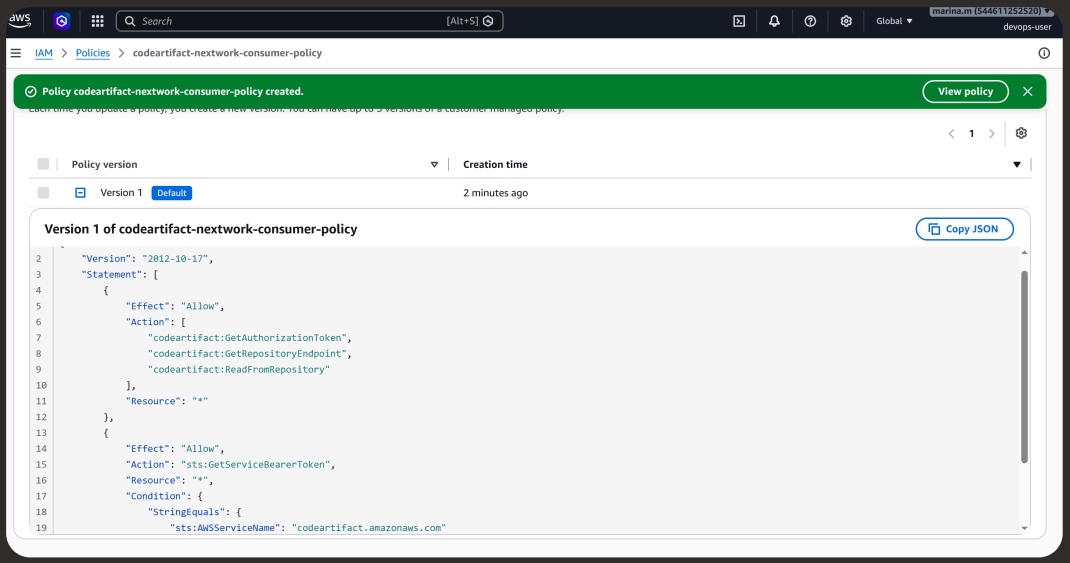
Resolution

I resolved the permissions error by creating an IAM policy with CodeArtifact permissions, creating an IAM role with that policy attached, and then attaching the role to my EC2 instance.

Using IAM roles is a security best practice because they give AWS resources temporary permissions without requiring us to store or hard-code access keys or passwords on the EC2 instance.

The JSON policy attached to my role

This IAM policy allows the EC2 instance to authenticate with CodeArtifact, find the repository endpoint, and read/download packages, while allowing AWS STS to provide the required service token for CodeArtifact access.

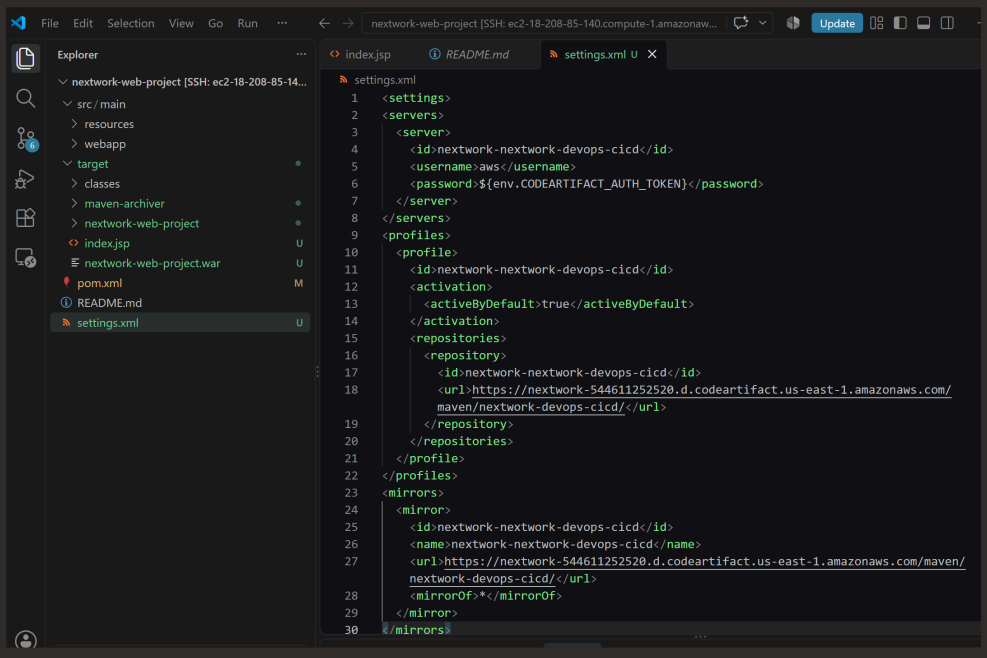


Maven and CodeArtifact

To test the connection between Maven and CodeArtifact, I compiled my web app using settings.xml

The settings.xml file tells Maven where the CodeArtifact repository is located and how to authenticate with it, allowing Maven to download and manage project dependencies through CodeArtifact.

Compiling means converting your Java source code into a format that the computer can understand and run.



Verify Connection

I checked the nextwork-devops-cicd AWS CodeArtifact repository after compiling and deploying my web app. I saw my package nextwork-web-project under the namespace com.nextwork.app, with version 1.0-SNAPSHOT and a Published status. This confirmed that my web application was successfully published to CodeArtifact.

